

Broadcast e Multicast

- **Broadcast**

Invio di pacchetti a *tutti* i nodi di una rete locale. Non richiede iscrizione: ogni host sulla subnet li riceve.

- **Multicast**

Invio di pacchetti a un *gruppo selezionato* di destinatari che si iscrivono a uno specifico indirizzo multicast.

- Il **broadcast** esiste solo in IPv4 e solo con pacchetti non orientati alla connessione (UDP)
- Il multicast **esiste solo su protocolli datagram** (UDP)
- In **IPv6 non esiste il broadcast**. È stato **eliminato** e sostituito dal multicast.

Broadcast

- **Broadcast**

Invio di pacchetti a *tutti* i nodi di una rete locale. Non richiede iscrizione: ogni host sulla subnet li riceve.

- Indirizzo usato è il broadcast della subnet, es.: 192.168.1.255 per la rete 192.168.1.0/24 oppure broadcast globale 255.255.255.255 (spesso bloccato dai router).

Sender (mittente):

1. `socket(AF_INET, SOCK_DGRAM, 0)`
2. `setsockopt(..., SO_BROADCAST, ...)` → abilitato esplicitamente
3. Mandare il pacchetto a `xxx.xxx.xxx.255:porta` con `sendto()`

Receiver (ricevitore):

1. `socket(AF_INET, SOCK_DGRAM, 0)`
2. `bind()` su:
 1. `INADDR_ANY`
 2. e la **stessa porta**
3. Ricevere con `recvfrom()`

Broadcast

▪ Sender

```
#define BROADCAST_ADDR "255.255.255.255" //broadcast limitato
```

```
int main() {
    int sock; struct sockaddr_in addr; char *msg = "Hello broadcast!";
    // 1. Crea socket UDP
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) { perror("socket"); exit(1);}

    // 2. Abilita il broadcast
    int yes = 1;
    if (setsockopt(sock, SOL_SOCKET, SO_BROADCAST, &yes, sizeof(yes)) < 0) {
        perror("SO_BROADCAST"); exit(1); }

    // 3. Imposta indirizzo broadcast destinazione
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET; addr.sin_port = htons(PORT);
    addr.sin_addr.s_addr = inet_addr(BROADCAST_ADDR);

    // 4. Invia periodicamente
    while (1) {
        if (sendto(sock, msg, strlen(msg), 0, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
            perror("sendto"); exit(1);}
        printf("Inviato broadcast: %s\n", msg); sleep(1);
    }
    return 0; }
```

Broadcast

- **Receiver**

```
int main() {
    int sock;
    struct sockaddr_in addr;
    char buffer[1024];

    // 1. Socket UDP
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) {
        perror("socket");
        exit(1);
    }

    // 2. Permetti riuso addr e porta
    int yes = 1;
    setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));

    // 3. Bind su tutte le interfacce
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(PORT);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

Broadcast

▪ Receiver

```
if (bind(sock, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
    perror("bind");
    exit(1);
}

printf("In ascolto broadcast UDP sulla porta %d...\n", PORT);

// 4. Ricezione loop
while (1) {
    int n = recv(sock, buffer, sizeof(buffer)-1, 0);
    if (n < 0) {
        perror("recv");
        exit(1);
    }
    buffer[n] = '\0';
    printf("Ricevuto: %s\n", buffer);
}

return 0;
}
```

Broadcast

Broadcast “limitato” (255.255.255.255)

- Funziona sempre in locale, indipendentemente dalla rete.
- Perfetto per test iniziali del codice UDP.
- Il pacchetto rimane nella rete locale dell'interfaccia (non attraversa router o gateway virtuali).

Broadcast di subnet

- È il vero broadcast della rete (es. 192.168.29.255).
- Arriva a tutti gli host nella stessa subnet (LAN reale o rete virtuale VMware).
- Dipende dall'indirizzo assegnato dal DHCP della macchina/VM.

Usiamo il comando:

ip link	%cercare interfaccia rete (es. ens33)
	%poi
ip addr show ens33	%cercare IP dell'interfaccia
inet 192.168.29.128/24 brd 192.168.29.255 scope global ens33	

Broadcast

Broadcast di subnet

- È il vero broadcast della rete (es. 192.168.29.255).
- Arriva a tutti gli host nella stessa subnet (LAN reale o rete virtuale VMware).
- Dipende dall'indirizzo assegnato dal DHCP della macchina/VM.

NAT (Network Address Translation)

Tecnica che mappa gli indirizzi di una rete privata verso l'esterno usando l'indirizzo del router. Il router NAT riscrive gli indirizzi sorgente/destinazione dei pacchetti.

Attenzione: NAT non inoltra broadcast né multicast, quindi il traffico broadcast resta sempre dentro la rete privata (incluso il NAT virtuale di VMware)

Modalità Bridged

La VM entra nella rete reale: riceve un IP dal router fisico (es. 192.168.1.x).

In questa modalità il broadcast di subnet (es. 192.168.1.255) può raggiungere tutti i PC della LAN reale.

Attenzione: su Wi-Fi il bridging può essere limitato dai driver: alcuni broadcast potrebbero non essere inoltrati alla VM.

Multicast

Il **multicast** è un meccanismo in cui un processo invia un pacchetto UDP a un gruppo di destinatari identificati da un indirizzo IP speciale.

I processi che vogliono ricevere questi pacchetti **devono iscriversi al gruppo (join)** tramite la socket option: `IP_ADD_MEMBERSHIP`

Indirizzi IPv4 multicast: 224–239.x.x.x

Sottoclassi note:

- 224.0.0.x link-local (non escono dalla LAN)
- 239.x.x.x amministrative locali (consigliate per applicazioni locali)
- 225–238 global scope (raramente usate senza configurazione router)

Per esercizi didattici: **239.x.x.x**

Multicast

Il **multicast** è un meccanismo in cui un processo invia un pacchetto UDP a un gruppo di destinatari identificati da un indirizzo IP speciale.

I processi che vogliono ricevere questi pacchetti **devono iscriversi al gruppo (join)** tramite la socket option: `IP_ADD_MEMBERSHIP`

Per ricevere il multicast il **receiver** deve:

- aprire socket UDP
- fare bind sulla porta
- iscriversi con:

```
struct ip_mreq mreq;          // per fare join a gruppo multicast
mreq.imr_multiaddr.s_addr = inet_addr("239.255.0.1"); //es. indirizzo gruppo
mreq.imr_interface.s_addr = htonl(INADDR_ANY); //indirizzo interfaccia locale
```

```
// iscrizione al gruppo
setsockopt(sock, IPPROTO_IP, IP_ADD_MEMBERSHIP, &mreq, sizeof(mreq));
```

Multicast

Il sender multicast invia pacchetti UDP a un indirizzo del range 239.x.x.x (administratively scoped), valido all'interno della LAN locale.

Per mandare il multicast il sender deve:

- creare socket UDP semplice
- impostare il TTL (numero massimo di hop)
- definire indirizzo di destinazione

```
sock = socket(AF_INET, SOCK_DGRAM, 0);
```

```
unsigned char ttl = 1; // multicast solo nella LAN, non attraversa router  
setsockopt(sock, IPPROTO_IP, IP_MULTICAST_TTL, &ttl, sizeof(ttl));
```

```
addr.sin_family = AF_INET;  
addr.sin_port   = htons(PORTA);  
addr.sin_addr.s_addr = inet_addr("239.255.0.1");
```

```
sendto(sock, buf, len, 0, (struct sockaddr *)&addr, sizeof(addr));
```

Multicast

II sender multicast

```
#define GROUP "239.255.0.1"
#define PORT 5000

int main() {
    int sock;
    struct sockaddr_in addr;
    unsigned char ttl = 1; // TTL = 1 → LAN locale

    // Socket UDP
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) {
        perror("socket");
        exit(1);
    }

    // Imposta TTL multicast
    if (setsockopt(sock, IPPROTO_IP, IP_MULTICAST_TTL,
                  &ttl, sizeof(ttl)) < 0) {
        perror("setsockopt - TTL");
        exit(1);
    }
}
```

Multicast

Il sender multicast

```
// Indirizzo multicast di destinazione
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port   = htons(PORT);
addr.sin_addr.s_addr = inet_addr(GROUP);

// Loop di invio
while (1) {
    char msg[128];
    sprintf(msg, "Ciao dalla VM multicast!");

    if (sendto(sock, msg, strlen(msg), 0,
               (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("sendto");
        exit(1);
    }
    printf("Inviato: %s\n", msg);
    sleep(1);
}
return 0;
}
```

Multicast

II receiver multicast

```
#define GROUP "239.255.0.1"
#define PORT 5000

int main() {
    int sock;
    struct sockaddr_in addr;
    struct ip_mreq mreq;
    char msgbuf[256];

    // Socket UDP
    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) {
        perror("socket");
        exit(1);
    }
    // Per permettere a più client di fare bind sullo stesso addr
    int reuse = 1;

    setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));
    setsockopt(sock, SOL_SOCKET, SO_REUSEPORT, &reuse, sizeof(reuse));
```

Multicast

II receiver multicast

```
memset(&addr, 0, sizeof(addr)); // Bind sulla porta (0.0.0.0:PORT)
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = htonl(INADDR_ANY);
addr.sin_port = htons(PORT);

if (bind(sock, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
    perror("bind"); exit(1);}

// Join del gruppo multicast
mreq.imr_multiaddr.s_addr = inet_addr(GROUP);
mreq.imr_interface.s_addr = htonl(INADDR_ANY);
if (setsockopt(sock, IPPROTO_IP, IP_ADD_MEMBERSHIP,
               &mreq, sizeof(mreq)) < 0) {
    perror("setsockopt - IP_ADD_MEMBERSHIP"); exit(1); }

while (1) { // Ricezione
    int n = recv(sock, msgbuf, sizeof(msgbuf)-1, 0);
    if (n < 0) { perror("recv"); exit(1);}
    msgbuf[n] = '\0'; printf("Ricevuto: %s\n", msgbuf);
}
return 0;
}
```

Multicast

IGMP JOIN: come un host entra in un gruppo multicast

- **IGMP (Internet Group Management Protocol):** È il protocollo con cui un host annuncia al router locale di voler ricevere un gruppo multicast.
- **Processo di JOIN (host → router)**
 - L'applicazione invoca `IP_ADD_MEMBERSHIP`
 - Il kernel invia un IGMP Report al gruppo (JOIN)
 - Il router registra che su quella sottorete ci sono membri del gruppo
 - Il router inizierà a richiedere il flusso (upstream) tramite **PIM (Protocolo Independent Multicast)**

[Host] --IGMP--> [Router A] --PIM--> [Router B] --PIM--> [Router C] ← Sorgente

Permette alla rete di sapere chi vuole cosa, evitando di inviare multicast a chi non è interessato

Multicast

PIM: come i router distribuiscono il multicast tra sottoreti

- **PIM (Protocol Independent Multicast)**

È il protocollo con cui i router costruiscono l'albero di distribuzione dei flussi multicast tra le sottoreti

- **Cosa fa PIM**

- decide su quali interfacce inviare il flusso multicast
- collega tra loro più router formando un albero
- aggiorna l'albero in base ai JOIN/LEAVE IGMP delle sottoreti

- **Cosa NON fa PIM**

non vede gli host singoli (non può filtrare indirizzi IP di host) controlla solo interfacce / sottoreti, non LAN interne

- **Scopo**

Propagare i pacchetti multicast **solo verso le sottoreti che li hanno richiesti**, evitando traffico inutile.

Multicast

Quando NON ci sono router o switch: multicast “senza controllo”

- **Nessuno interpreta IGMP**

In una rete senza router multicast e senza switch con IGMP snooping:

- gli IGMP Report **NON vengono letti da nessuno**
- non esiste una membership list nella rete
- non si formano alberi PIM (non ci sono router)

- **Risultato sulla distribuzione**

- mezzo fisico (bus/hub/Wi-Fi) copia tutti i pacchetti multicast a tutti gli host
- JOIN serve solo al kernel per decidere se consegnare o scartare il pacchetto

- **Conclusione**

Il multicast si comporta come un broadcast esteso, senza filtraggio fisico

Multicast

Il ruolo degli switch: IGMP Snooping

- **Che cos'è IGMP Snooping?**

Lo switch “spia” (snoop) i messaggi IGMP tra host e router, anche se non è un partecipante ufficiale

- **Cosa fa lo switch con lo snooping**

- costruisce una tabella porta ↔ gruppo
- inoltra ogni gruppo multicast **solo alle porte interessate**
- evita di inondare la rete con traffico inutilmente
- può filtrare host specifici (su base porta o IP/ACL)

- **Scopo**

Ottimizzare il multicast nella LAN:

- **Multicast → solo alle porte con host iscritti**
- **Niente più broadcast verso tutta la LAN**

Multicast

In sintesi ...

- **Host:**
Fa JOIN/LEAVE (IGMP) e riceve il traffico desiderato.
- **Router (PIM):**
 - Costruisce l'albero multicast tra sottoreti
 - Gestisce dove far passare il flusso
- **Switch (IGMP Snooping):**
 - Filtra la distribuzione nella LAN.
 - Decide quali porte ricevono il flusso
- **Reti senza router/switch:**
Nessun controllo

Multicast

Come mandare messaggi a destinatari selezionati?

- Il multicast IP non è sicuro su reti aperte
- Soluzione tipica - cifratura:
 - Cifrato il payload
 - Solo chi possiede la chiave può interpretarlo
 - Gli altri vedono solo pacchetti UDP non utilizzabili

Esercizio

Implementare una chat di gruppo via multicast

- Ogni peer può sia mandare che ricevere messaggi
- Non c'è server centrale, ogni peer è sia server che client
- Usare un unico socket UDP per peer
- Usare select per gestire stdin + socket
- Ogni peer:
 - Si iscrive ad uno stesso gruppo (es. 239.1.2.3:5000)
 - Ascolta su socket UDP
 - Legge da tastiera (stdin)
 - Scrive a tutto il gruppo i messaggi letti da stdin
 - Scrive in stdout i messaggi che riceve via socket